

Numerical Optimization in Python

Theory, Algorithms, Machine Learning, and Practical Implementation

Dr. Aurelle TCHAGNA

Computer Engineering Department, National Advanced School of Engineering-University of Yaounde 1, Cameroon.
MCTR, Department of Medicine, McMaster University, Hamilton, ON, Canada

January 16, 2026

Learning Outcomes

By the end of this course, students should be able to:

- Formulate unconstrained and constrained optimization problems.
- Understand gradients, Hessians, curvature, and optimality conditions.
- Identify convexity and explain why it matters for guarantees.
- Implement key algorithms: Gradient Descent, Newton, (L-)BFGS.
- Solve constrained problems using Lagrangian and KKT conditions.
- Use `scipy.optimize` effectively and interpret solver outputs.

Why Optimization?

Optimization is the engine behind:

- **Machine learning**: training = minimizing a loss.
- **Engineering**: design, calibration, control, parameter fitting.
- **Scientific computing**: inverse problems, model estimation.

Typical objective functions:

$$\min_{x \in \mathbb{R}^n} f(x) \quad (\text{energy} / \text{cost} / \text{loss})$$

Unconstrained optimization:

$$\min_{x \in \mathbb{R}^n} f(x)$$

Constrained optimization:

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) \\ \text{s.t.} \quad & g_i(x) \leq 0, \quad i = 1, \dots, m \\ & h_j(x) = 0, \quad j = 1, \dots, p \end{aligned}$$

Least squares (linear regression):

$$\min_{w \in \mathbb{R}^d} \|Xw - y\|_2^2$$

Ridge regression (L2):

$$\min_{w \in \mathbb{R}^d} \|Xw - y\|_2^2 + \lambda \|w\|_2^2$$

Lasso regression (L1):

$$\min_{w \in \mathbb{R}^d} \|Xw - y\|_2^2 + \lambda \|w\|_1$$

Ridge and Lasso regressions are optimization techniques that modify standard linear regression by adding penalty terms to the cost function, preventing overfitting and handling multicollinearity; Ridge uses an L2 penalty (squared coefficients) for continuous shrinkage, keeping all features, while Lasso uses an L1 penalty (absolute coefficients) for feature selection, potentially zeroing out less important coefficients for simpler models.

Differentiability and Smoothness

We often assume:

- f is differentiable: gradient $\nabla f(x)$ exists.
- f is twice differentiable: Hessian $\nabla^2 f(x)$ exists.
- f is L -smooth: ∇f is Lipschitz continuous:

$$\|\nabla f(x) - \nabla f(y)\| \leq L \|x - y\|.$$

Smoothness helps derive step-size rules and convergence rates.

Smoothness and Lipschitz Continuity (Intuition)

A function is called **smooth** if its gradient does not change too fast.

Lipschitz Gradient (L-smoothness)

A differentiable function f is **L-smooth** if:

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\| \quad \forall x, y$$

Intuition:

- The slope cannot suddenly become very steep
- The function has *no sharp corners*
- This guarantees **stable steps in gradient descent**

Example:

$$\begin{aligned} f(x) &= x^2 \\ \nabla f(x) &= 2x \end{aligned}$$

Then:

$$|2x - 2y| = 2|x - y| \Rightarrow f \text{ is } L\text{-smooth with } L = 2$$

Gradient: Definition and Meaning

Gradient:

$$\nabla f(x) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix}$$

Geometric interpretation:

- Direction of steepest **increase**.
- $-\nabla f(x)$ is the steepest **descent** direction.
- Level sets: $\nabla f(x)$ is orthogonal to $f(x) = c$.

Hessian:

$$\nabla^2 f(x) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Local quadratic approximation:

$$f(x + \Delta) \approx f(x) + \nabla f(x)^T \Delta + \frac{1}{2} \Delta^T \nabla^2 f(x) \Delta$$

Critical Points and Curvature Test

A point x is **critical** if:

$$\nabla f(x)=0$$

Second-order test:

- If $\nabla^2 f(x^*) \succ 0$ (positive definite) $\Rightarrow$ strict local minimum.
- If $\nabla^2 f(x^*) \prec 0 \Rightarrow$ strict local maximum.
- If $\nabla^2 f(x^*)$ is indefinite $\Rightarrow$ saddle point.

Convexity: Definition

f is convex if:

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y), \quad \forall x, y, \theta \in [0, 1].$$

Key consequences:

- Any local minimum is a global minimum.
- Optimization is easier: strong guarantees and stable algorithms.

Convexity: Hessian Criterion

If f is twice differentiable:

$$f \text{ convex} \iff \nabla^2 f(x) \succeq 0 \quad \forall x.$$

If strongly convex ($\mu > 0$):

$$\nabla^2 f(x) \succeq \mu I \implies \text{unique global minimizer.}$$

Optimality Conditions (Unconstrained)

First-order necessary condition:

$$\nabla f(x^*) = 0$$

Second-order sufficient condition:

$$\nabla f(x^*) = 0 \text{ and } \nabla^2 f(x^*) \succ 0 \Rightarrow x^* \text{ is a local minimum.}$$

If f is convex, the first-order condition is also sufficient.

Convexity: Simple Real Application

Application: A clinic wants to choose a daily stock level x of a medicine (e.g., malaria test kits). If the clinic stocks too many, storage cost increases; if it stocks too few, shortage cost increases.

A simple convex cost model is:

$$C(x) = (x - 50)^2 + 200$$

where x is the number of kits, and 50 is the target demand.

Why is $C(x)$ convex? Second derivative test (1D):

$$C'(x) = 2(x - 50), \quad C''(x) = 2 > 0$$

So $C(x)$ is **strictly convex** (one global minimum).

Solution (optimal stock):

$$C'(x) = 0 \Rightarrow 2(x - 50) = 0 \Rightarrow \boxed{x^* = 50}$$

Interpretation: Convexity guarantees the best decision is unique and easy to find.

Exercise: Check Convexity (One Convex, One Non-Convex)

Exercise: Determine whether each function is convex on $\mathbb{R}$.

① $f_1(x) = x^2 + 4x + 1$

② $f_2(x) = x^4 - 4x^2$

Hint (1D test): A twice differentiable function is convex if:

$$f''(x) \geq 0 \quad \forall x$$

and non-convex if $f''(x)$ becomes negative somewhere.

Question: Which one is convex? Which one is non-convex?

Solution: Convex vs Non-Convex

1) $f_1(x) = x^2 + 4x + 1$

$$f_1'(x) = 2x + 4, \quad f_1''(x) = 2$$

Since $f_1''(x) = 2 > 0$ for all x , we conclude:

f_1 is strictly convex on $\mathbb{R}$.

2) $f_2(x) = x^4 - 4x^2$

$$f_2'(x) = 4x^3 - 8x, \quad f_2''(x) = 12x^2 - 8$$

Now check where $f_2''(x) < 0$:

$$12x^2 - 8 < 0 \Rightarrow x^2 < \frac{2}{3} \Rightarrow |x| < \sqrt{\frac{2}{3}}$$

So $f_2''(x)$ is negative near $x = 0$, therefore:

f_2 is non-convex on $\mathbb{R}$.

Interpretation: Non-convex functions can have multiple local minima, making optimization harder

Convex vs Non-Convex: Visual Intuition

Convex function

- Shape: bowl
- One global minimum
- Any local minimum is global
- Gradient descent always converges

Non-convex function

- Shape: valleys and hills
- Multiple local minima
- Can trap optimization algorithms
- Initialization matters

Geometric definition: A function f is convex if for all x, y and $\lambda \in [0, 1]$:

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$

Gradient Descent: Algorithm

Update rule:

$$x_{k+1} = x_k - \alpha_k \nabla f(x_k)$$

Key components:

- Step size α_k (constant or adaptive).
- Stopping criteria: $\|\nabla f(x_k)\| \leq \varepsilon$ or small improvement.

Gradient Descent: Worked Example

Problem:

$$\min_{x \in \mathbb{R}} f(x) = (x - 4)^2$$

The derivative is:

$$f'(x) = 2(x - 4)$$

Gradient descent update:

$$x_{k+1} = x_k - \alpha f'(x_k) = x_k - \alpha 2(x_k - 4)$$

Let $\alpha = 0.25$ and start at $x_0 = 0$.

$$x_1 = 0 - 0.25(-8) = 2$$

$$x_2 = 2 - 0.25(-4) = 3$$

$$x_3 = 3 - 0.25(-2) = 3.5$$

The iterates move toward the minimum at $x^* = 4$.

Exercise: Gradient Descent in 1D

Exercise:

$$\min_{x \in \mathbb{R}} f(x) = (x - 1)^2$$

Derivative:

$$f'(x) = 2(x - 1)$$

Use gradient descent with $\alpha = 0.5$ and $x_0 = 4$.

Solution:

$$x_1 = 4 - 0.5(6) = 1$$

Since $x_1 = 1$, which is the minimum, the algorithm converges in one step.

$$x^* = 1$$

Gradient Descent: Step Size Choices

Fixed step: $\alpha_k = \alpha$

- Simple but sensitive (may diverge if too large).

Line search:

- Choose α_k to satisfy sufficient decrease (Armijo) and curvature (Wolfe).

For L -smooth convex f , a common safe choice: $\alpha \leq \frac{1}{L}$.

Why Smoothness Matters for Gradient Descent

Consider gradient descent:

$$x_{k+1} = x_k - \alpha \nabla f(x_k)$$

If f is L -smooth, then choosing:

$$0 < \alpha \leq \frac{1}{L}$$

guarantees:

- The objective decreases at every iteration
- No oscillations or divergence

Concrete example:

$$f(x) = x^2 \quad \Rightarrow \quad L = 2$$

Safe step size:

$$\alpha \leq 0.5$$

- $\alpha = 0.4 \rightarrow$ smooth convergence
- $\alpha = 1 \rightarrow$ oscillations
- $\alpha > 1 \rightarrow$ divergence

Convergence (Key Rates)

For convex L -smooth functions:

$$f(x_k) - f(x^*) = \mathcal{O}\left(\frac{1}{k}\right)$$

For μ -strongly convex and L -smooth functions:

$$f(x_k) - f(x^*) = \mathcal{O}\left(\left(1 - \frac{\mu}{L}\right)^k\right)$$

Interpretation:

- Strong convexity gives **linear** convergence.
- Conditioning $\kappa = L/\mu$ affects speed.

Momentum and Nesterov Acceleration

Momentum:

$$v_{k+1} = \beta v_k + \nabla f(x_k), \quad x_{k+1} = x_k - \alpha v_{k+1}$$

Why it helps:

- Reduces oscillations in narrow valleys.
- Accelerates convergence in ill-conditioned problems.

Nesterov: evaluates gradient at a look-ahead point (often faster).

Momentum and Nesterov: Simple Intuition

Analogy: Ball rolling down a valley

- Imagine a ball rolling down a long, narrow valley.
- Gradient descent = the ball stops and re-evaluates direction at every step.
- Momentum = the ball keeps part of its speed from previous steps.

Momentum update:

$$v_{k+1} = \beta v_k + \nabla f(x_k), \quad x_{k+1} = x_k - \alpha v_{k+1}$$

Effect:

- Previous directions accumulate (inertia).
- Side-to-side oscillations are damped.
- Faster movement along the valley floor.

Nesterov idea:

- Look slightly ahead before deciding direction.
- Anticipates where the ball is going.
- Often converges faster than classical momentum.

Newton's Method

Newton step solves:

$$\nabla^2 f(x_k) \Delta_k = \nabla f(x_k)$$

Update:

$$x_{k+1} = x_k - \Delta_k = x_k - [\nabla^2 f(x_k)]^{-1} \nabla f(x_k)$$

Properties:

- Quadratic convergence near the optimum.
- Expensive for large n (Hessian + linear solve).

Quasi-Newton (BFGS / L-BFGS)

Idea: approximate inverse Hessian $H_k \approx [\nabla^2 f(x_k)]^{-1}$.

$$x_{k+1} = x_k - \alpha_k H_k \nabla f(x_k)$$

BFGS:

- Very effective in practice (smooth problems).

L-BFGS:

- Memory-efficient for large-scale problems (stores few vectors).

Quasi-Newton (BFGS / L-BFGS): Application Example

BFGS (Broyden–Fletcher–Goldfarb–Shanno) and L-BFGS (Limited-memory BFGS) are both popular quasi-Newton methods used for solving unconstrained nonlinear optimization problems.

Application: Parameter estimation in healthcare

Suppose we want to estimate parameters $\theta = (\beta, \gamma)$ of an epidemic model (SIR model for cholera or COVID) by minimizing the error:

$$J(\theta) = \sum_t (I_{\text{model}}(t; \theta) - I_{\text{data}}(t))^2$$

Problem characteristics:

- $J(\theta)$ is smooth but ill-conditioned
- Gradient descent is slow and zig-zags
- Computing the exact Hessian is expensive

Quasi-Newton idea (BFGS):

- Learn curvature information from past gradients
- Build an approximation of the inverse Hessian
- Automatically choose better directions than gradient descent

Quasi-Newton (BFGS / L-BFGS): Application Example

$$x_{k+1} = x_k - \alpha_k H_k \nabla J(x_k)$$

Why L-BFGS?

- Same idea, but stores only a few vectors
- Scales to large ML models and datasets
- Used in practice for logistic regression and deep learning

Lagrangian for Constraints

For:

$$\min_x f(x) \text{ s.t. } g_i(x) \leq 0, \quad h_j(x) = 0$$

Lagrangian:

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^m \lambda_i g_i(x) + \sum_{j=1}^p \nu_j h_j(x)$$

$\lambda_i \geq 0$ are multipliers for inequality constraints.

Exercise: Build the Lagrangian

Exercise:

Consider the constrained optimization problem:

$$\min_{x \in \mathbb{R}} f(x) = (x - 3)^2$$

subject to:

$$g(x) = 2 - x \leq 0 \quad (\text{i.e. } x \geq 2)$$

Questions:

- 1 Identify $f(x)$ and $g(x)$.
- 2 Write the Lagrangian $\mathcal{L}(x, \lambda)$.
- 3 State the sign condition on λ .

Hint:

$$\mathcal{L}(x, \lambda) = f(x) + \lambda g(x)$$

Correction: Lagrangian Construction

Step 1: Identify components

Objective:

$$f(x) = (x - 3)^2$$

Constraint:

$$g(x) = 2 - x \leq 0$$

Step 2: Build the Lagrangian

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(2 - x)$$

Step 3: Multiplier condition

Since $g(x) \leq 0$, the multiplier must satisfy:

$$\lambda \geq 0$$

Interpretation: λ measures how strongly the constraint $x \geq 2$ affects the solution.

Correction: Lagrangian Construction

Step 1: Identify components

Objective:

$$f(x) = (x - 3)^2$$

Constraint:

$$g(x) = 2 - x \leq 0$$

Step 2: Build the Lagrangian

$$\mathcal{L}(x, \lambda) = (x - 3)^2 + \lambda(2 - x)$$

Step 3: Multiplier condition

Since $g(x) \leq 0$, the multiplier must satisfy:

$$\lambda \geq 0$$

Interpretation: λ measures how strongly the constraint $x \geq 2$ affects the solution.

KKT Conditions

The optimality conditions for a constrained local optimum are called the Karush Kuhn Tucker (KKT) conditions and they play an important role in constrained optimization theory and algorithm development. The KKT conditions for optimality are a set of necessary conditions for a solution to be optimal in a mathematical optimization problem. They are necessary and sufficient conditions for a local minimum in nonlinear programming problems. KKT conditions:

- ❶ **Stationarity:** $\nabla_x \mathcal{L}(x^*, \lambda^*, \nu^*) = 0$
- ❷ **Primal feasibility:** $g_i(x^*) \leq 0, \quad h_j(x^*) = 0$
- ❸ **Dual feasibility:** $\lambda_i^* \geq 0$
- ❹ **Complementary slackness:** $\lambda_i^* g_i(x^*) = 0$

For convex problems with regularity conditions, KKT is sufficient for global optimality.

KKT Example (Worked): Quadratic with One Inequality

Problem:

$$\min_{x \in \mathbb{R}} f(x) = (x - 2)^2 \quad \text{s.t.} \quad g(x) = 1 - x \leq 0 \quad (\text{i.e. } x \geq 1).$$

Lagrangian:

$$\mathcal{L}(x, \lambda) = (x - 2)^2 + \lambda(1 - x), \quad \lambda \geq 0.$$

KKT conditions:

- Stationarity: $\frac{d\mathcal{L}}{dx} = 2(x - 2) - \lambda = 0$
- Primal feasibility: $1 - x \leq 0 \Rightarrow x \geq 1$
- Dual feasibility: $\lambda \geq 0$
- Complementary slackness: $\lambda(1 - x) = 0$

Solve: Unconstrained minimizer is $x = 2$, which is feasible ($2 \geq 1$). So the constraint is inactive $\Rightarrow \lambda = 0$. Stationarity gives $2(x - 2) = 0 \Rightarrow x^* = 2$, and $\lambda^* = 0$.

Exercise : KKT for a Simple Bound Constraint

Exercise:

$$\min_{x \in \mathbb{R}} f(x) = x^2 \quad \text{s.t.} \quad g(x) = 2 - x \leq 0 \quad (\text{i.e. } x \geq 2).$$

Exercise : KKT for a Simple Bound Constraint

Solution:

$$\min_{x \in \mathbb{R}} f(x) = x^2 \quad \text{s.t.} \quad g(x) = 2 - x \leq 0 \quad (\text{i.e. } x \geq 2).$$

Step 1: Lagrangian

$$\mathcal{L}(x, \lambda) = x^2 + \lambda(2 - x), \quad \lambda \geq 0.$$

Step 2: KKT conditions

$$\text{Stationarity: } 2x - \lambda = 0$$

$$\text{Primal: } x \geq 2, \quad \text{Dual: } \lambda \geq 0, \quad \text{Slackness: } \lambda(2 - x) = 0.$$

Solution: The unconstrained minimizer $x = 0$ is **not feasible**. Thus optimum is at the boundary $x^* = 2$ (active constraint). From stationarity: $2x^* - \lambda^* = 0 \Rightarrow \lambda^* = 4$.

$$x^* = 2, \quad \lambda^* = 4, \quad f(x^*) = 4.$$

Example: KKT

Problem:

$$\min_{x \in \mathbb{R}} f(x) = x^3 \quad \text{s.t.} \quad g(x) = x^2 \leq 0$$

Example: KKT Cannot Be Satisfied (Non-Convex Problem)

Problem:

$$\min_{x \in \mathbb{R}} f(x) = x^3 \quad \text{s.t.} \quad g(x) = x^2 \leq 0$$

Step 1: Feasible set

Since $x^2 \leq 0$, the only feasible point is:

$$x = 0$$

Step 2: Lagrangian

$$\mathcal{L}(x, \lambda) = x^3 + \lambda x^2, \quad \lambda \geq 0$$

Step 3: KKT stationarity

$$\nabla_x \mathcal{L}(x, \lambda) = 3x^2 + 2\lambda x$$

At $x = 0$:

$$\nabla_x \mathcal{L}(0, \lambda) = 0 \quad \text{for all } \lambda$$

But: $f(x) = x^3$ is **non-convex** and has no minimum around $x = 0$.

Conclusion: This problem violates regularity conditions and is non-convex; KKT does not characterize optimality.

Why KKT Fails Here (Key Insight)

- The objective $f(x) = x^3$ is **non-convex**
- The feasible set collapses to a single point
- There is no local minimum (only an inflection point)
- Gradient information is insufficient

Important principle:

If a problem is convex and satisfies regularity conditions, then KKT conditions are **necessary and sufficient**.

Here:

- Problem is non-convex
- Regularity conditions fail
- KKT provides no meaningful optimality certificate

Teaching takeaway: Failure or ambiguity of KKT is a strong warning sign of non-convexity or irregular constraints.

Non-Smooth Optimization and Proximal Operators

Many objectives are non-smooth, e.g. Lasso:

$$\min_x \underbrace{f(x)}_{\text{smooth}} + \lambda \|x\|_1$$

Proximal operator:

$$\text{prox}_\lambda(z) = \arg \min_x \left\{ \frac{1}{2} \|x - z\|_2^2 + \lambda \|x\|_1 \right\}$$

For ℓ_1 , prox is soft-thresholding:

$$(\text{prox}_\lambda(z))_i = \text{sign}(z_i) \max(|z_i| - \lambda, 0)$$

ISTA for $\min f(x) + \lambda \|x\|_1$:

$$x_{k+1} = \text{prox}_{\alpha\lambda}(x_k - \alpha \nabla f(x_k))$$

FISTA accelerates ISTA with momentum-like extrapolation and achieves faster theoretical rate.

Implementation Pattern in Python

Common implementation structure:

- Define $f(x)$ and $\nabla f(x)$ (and optionally $\nabla^2 f(x)$).
- Choose algorithm + stopping criteria.
- Log metrics: objective value, gradient norm, iterations.
- Visualize convergence: $f(x_k)$ vs k .

Key function:

```
scipy.optimize.minimize( $f$ ,  $x_0$ , method, jac, hess, bounds, constraints)
```

Popular methods:

- BFGS: unconstrained, smooth
- L-BFGS-B: box constraints
- SLSQP: general constraints
- trust-constr: robust constrained solver

SciPy Example: Unconstrained (BFGS)

```
import numpy as np
from scipy.optimize import minimize

def f(x):
    return (1-x[0])**2 + 100*(x[1]-x[0]**2)**2    # Rosenbrock

def grad_f(x):
    dfdx = -2*(1-x[0]) - 400*x[0]*(x[1]-x[0]**2)
    dfdy = 200*(x[1]-x[0]**2)
    return np.array([dfdx, dfdy])

x0 = np.array([-1.2, 1.0])
res = minimize(f, x0, jac=grad_f, method="BFGS")
print(res.x, res.fun, res.nit)
```

SciPy Example: Box Constraints (L-BFGS-B)

```
import numpy as np
from scipy.optimize import minimize

def f(x):
    return (x[0]-2)**2 + (x[1]+1)**2

def grad_f(x):
    return np.array([2*(x[0]-2), 2*(x[1]+1)])

bounds = [(-1, 3), (-2, 2)]
x0 = np.array([0.0, 0.0])

res = minimize(f, x0, jac=grad_f, method="L-BFGS-B", bounds=bounds)
print(res.x, res.fun)
```

SciPy Example: Constrained (SLSQP)

```
import numpy as np
from scipy.optimize import minimize

def f(x):
    return x[0]**2 + x[1]**2

def grad_f(x):
    return np.array([2*x[0], 2*x[1]])

# Constraint:  $x_0 + x_1 = 1$ 
cons = {"type": "eq", "fun": lambda x: x[0] + x[1] - 1}

x0 = np.array([0.5, 0.5])
res = minimize(f, x0, jac=grad_f, method="SLSQP", constraints=[cons])
print(res.x, res.fun)
```

Gradient Descent with Line Search (Pseudocode)

Goal: minimize $f(x)$

Input:

- Initial point x_0
- Tolerance $\varepsilon > 0$
- Maximum iterations K

Algorithm:

- 1 Set $k = 0$
- 2 **while** $\|\nabla f(x_k)\| > \varepsilon$ and $k < K$:
 - 1 Compute descent direction: $d_k = -\nabla f(x_k)$
 - 2 Find step size α_k using line search:

$$\alpha_k = \arg \min_{\alpha > 0} f(x_k + \alpha d_k)$$

- 3 Update:

$$x_{k+1} = x_k + \alpha_k d_k$$

- 4 $k \leftarrow k + 1$

Why Stochastic Gradient Descent (SGD)?

For large datasets:

$$f(x) = \frac{1}{N} \sum_{i=1}^N \ell(x; \xi_i)$$

Problems with full Gradient Descent:

- Expensive: gradient uses all N samples
- Slow for large-scale machine learning

Idea of SGD:

- Use a single sample (or mini-batch)
- Much cheaper updates
- Faster per-iteration progress

Stochastic Gradient Descent (SGD)

Update rule:

$$x_{k+1} = x_k - \alpha_k \nabla \ell(x_k; \xi_k)$$

where ξ_k is a randomly sampled data point.

Key properties:

- Noisy gradient estimate
- Much faster iterations
- Converges in expectation

Gradient Descent vs SGD

	GD	SGD
Gradient cost	$O(N)$	$O(1)$
Noise	No	Yes
Speed per iter	Slow	Fast
Convergence	Smooth	Noisy
Used in deep learning	Rarely	Yes

Practical SGD Variants

- Mini-batch SGD
- SGD + Momentum
- Adam, RMSProp, Adagrad

These methods dominate modern deep learning optimization.

Why Adaptive Gradient Methods?

Limitations of standard Gradient Descent and SGD:

- Same learning rate for all coordinates
- Poor performance on ill-conditioned problems
- Sensitive to feature scaling

Idea:

- Adapt learning rates per coordinate
- Reduce oscillations
- Improve robustness and convergence speed

RMSProp Algorithm

RMSProp adapts the learning rate using a running average of squared gradients.

Algorithm:

$$g_k = \nabla f(x_k)$$

$$v_k = \beta v_{k-1} + (1 - \beta) g_k^2$$

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{v_k} + \varepsilon} g_k$$

Parameters:

- α : base learning rate
- $\beta \in [0.9, 0.99]$: decay factor
- ε : numerical stability

RMSProp: Intuition

- Large gradients $\Rightarrow$ large $v_k \Rightarrow$ smaller step
- Small gradients $\Rightarrow$ larger effective step
- Automatically rescales directions

Effect:

- Stabilizes SGD
- Faster convergence on ravines
- Widely used in deep learning

Adam: Adaptive Moment Estimation

Adam combines:

- Momentum (first moment)
- RMSProp (second moment)

First moment (mean):

$$m_k = \beta_1 m_{k-1} + (1 - \beta_1) g_k$$

Second moment (variance):

$$v_k = \beta_2 v_{k-1} + (1 - \beta_2) g_k^2$$

Because m_k and v_k are initialized at zero, we apply bias correction:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}$$

$$\hat{v}_k = \frac{v_k}{1 - \beta_2^k}$$

Final update:

$$x_{k+1} = x_k - \frac{\alpha}{\sqrt{\hat{v}_k} + \epsilon} \hat{m}_k$$

Adam: Why It Is Popular

- Works well out-of-the-box
- Robust to noisy gradients
- Invariant to gradient scaling
- Dominates deep learning optimization

Typical parameters:

$$\alpha = 10^{-3}, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$$

Convergence of Gradient Descent (Convex Case)

Assume:

- f convex and differentiable
- ∇f is L -Lipschitz

With step size $\alpha \leq \frac{1}{L}$:

$$f(x_k) - f(x^*) \leq \frac{\|x_0 - x^*\|^2}{2\alpha k}$$

Rate:

$$f(x_k) - f(x^*) = \mathcal{O}\left(\frac{1}{k}\right)$$

Proof Sketch: Gradient Descent Convergence

Key inequality (smoothness):

$$f(y) \leq f(x) + \nabla f(x)^T (y - x) + \frac{L}{2} \|y - x\|^2$$

Set $y = x_k - \alpha \nabla f(x_k)$.

After rearranging and summing over k :

$$\sum_{k=0}^{K-1} (f(x_k) - f(x^*)) \leq \frac{\|x_0 - x^*\|^2}{2\alpha}$$

Divide by K to obtain $\mathcal{O}(1/K)$ rate.

Strongly Convex Case

If f is μ -strongly convex and L -smooth:

$$f(x_k) - f(x^*) \leq \left(1 - \frac{\mu}{L}\right)^k (f(x_0) - f(x^*))$$

Rate:

- Linear (exponential) convergence
- Condition number $\kappa = \frac{L}{\mu}$ controls speed

Convergence of SGD

Assume:

- Unbiased gradient estimates
- Bounded variance: $\mathbb{E}\|\nabla\ell(x, \xi)\|^2 \leq \sigma^2$

With diminishing step size $\alpha_k \sim \frac{1}{\sqrt{k}}$:

$$\mathbb{E}[f(x_k) - f(x^*)] = \mathcal{O}\left(\frac{1}{\sqrt{k}}\right)$$

SGD trades accuracy for speed and scalability.

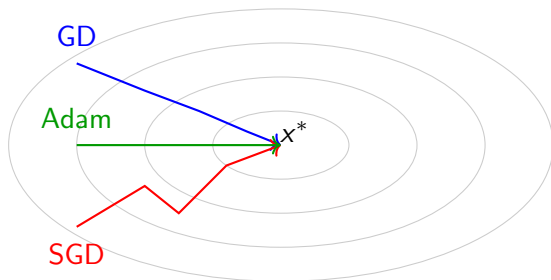
Interpreting Convergence Results

- GD: smooth, stable, slow per iteration
- SGD: noisy, fast, scalable
- Adam/RMSProp: heuristic but effective

Key lesson:

- Theory explains behavior
- Practice chooses algorithm

GD vs SGD vs Adam: Optimization Trajectories



Interpretation:

- GD: smooth but slow
- SGD: noisy but fast
- Adam: adaptive and stable

Practical Tips and Common Pitfalls

- **Feature scaling:** improves conditioning and convergence speed.
- **Step size:** too large diverges; too small is slow.
- **Stopping criteria:** use gradient norm and objective decrease.
- **Check solutions:** constraints satisfaction, sensitivity, stability.
- **Diagnostics:** plot objective and gradient norm vs iterations.

- Implement Gradient Descent + Line Search; compare with BFGS.
- Fit Ridge vs Lasso and discuss optimization differences.
- Compare convergence on ill-conditioned quadratic problems.
- Constrained portfolio optimization (box + sum-to-one constraint).

- Optimization problems: unconstrained vs constrained.
- Gradient/Hessian provide direction and curvature information.
- Convexity gives strong guarantees (global optimality).
- Algorithms: GD (first-order), Newton (second-order), BFGS (quasi-Newton).
- Constrained optimization: Lagrangian + KKT conditions.
- Python practice: SciPy solvers for real-world tasks.